Software Engineering at Google

- It’s programming if clever is a compliment, but it’s software engineering if clever is an accusation.
- Much of the difference between programming and software engineering stems from the management of code over time, the impact of time on scale, and decision making in the face of those ideas. Programming is the immediate act of producing code. Software engineering is the set of policies, practices, and tools that are necessary to make that code useful for as long as it needs to be used and allowing collaboration across a team. Software is sustainable when for the expected lifespan of the code we are capable of responding to changes in dependencies, technologies, and product requirements.
- Hyrum's Law is a software engineering observation stating that with a sufficient number of users of an API, all observable system behaviors will eventually be depended upon by someone, regardless of what is promised in the formal documentation or contract.
- “Shifting left.” The earlier in the development process a bug or security defect is caught, the less resources are required to fix it. A flaw released to production is expensive and difficult to fix. Compare this to catching it before the first commit so that no one has to go back and change it and you already have a developer familiar with the detail being implemented doing the work. That fact brings a lot of value to unit tests and static analysis tools.
- Software engineering is a team endeavor. Be humble, respect others, and trust others to be competent and do the right thing. Often social skills are more important than programming expertise.
- Lots on management. Some about being data-driven and meaningfully measuring engineering productivity. Code reviews.
- So much about testing. The highlights are breaking tests into small (unit/single process, no IO or networking), medium (integration tests; multithreaded but localhost only, fine to spin up a local server), and large (functional and end-to-end tests; anything goes). The larger a test gets the more likely they need to be automatically retried to deal with flakiness. 80% of an engineer’s test should be unit tests with some others to test system boundary interaction and emergent behavior. Remember the Beyoncé Rule: “If you liked it, you should’ve put a test on it.” (In other words if some change outside your software breaks your system, you should’ve put have had a test of the system relying on the expected behavior). Test should not rely on implementation details (ie. a rectangle test shouldn’t count on the rectangle being implemented as two triangles when that could be changed to being two smaller rectangles). The ideal test is written once and never needs to change

unless system requirements change. The best way to ensure this is writing tests that use the system the same way users would, through the public api. Mocking tends to create brittle integration tests, so prefer real objects. You want to test outcomes like “The system created a user” (for example, I can retrieve the user afterwards instead of testing the put method of the db was called for a User record.

- Tests should be clear and concise. A test is complete when its body contains all the information it needs for a reader to know how it arrived at a result. A test is concise when it contains no other distracting or irrelevant information. It can often be worth violating DRY if it leads to clearer tests. Test behaviors, not methods. A method can also do multiple things to be tested separately. A behavior is any guarantee a system makes about how it will respond to a set of inputs while it is in a particular state. Behaviors can often be expressed using “given,” “when,” and “then.” Most non-trivial methods implement multiple behaviors and it isn’t uncommon for a behavior to rely on the interaction of multiple methods; so this is a many-to-many relationship.
- When naming tests consider starting methods with “should” so it can be read as “Class under test” “should” behavior. If you are tempted to name a test method with “and”, you might need to split up your test.
- A good test failure message distinguishes between what is expected and what occurred.
- Test code should be DAMP (promote descriptive and meaningful phrases) rather than DRY (don’t repeat yourself). Since the goal of a test is to be stable, it is less valuable not to repeat yourself, and the cost of complexity is higher since test code isn’t tested. Helper methods should be implemented to hide irrelevant details and make the test more meaningful rather than reduce all repetition.
- Many engineers at Google avoid mocking frameworks or use them lightly. Originally, they were heavily used, but this led to brittle tests that didn’t catch many bugs. Google prefers to use real code when possible, which is referred to as classical testing. If the real code is too slow or is indeterministic, then it may prove useful to use a test double. When this is true, prefer a high fidelity fake if it is available. A good fake can save a lot of time in tests, but a good fake is most commonly provided and maintained by the owner of the API. An in-memory database can be a good fake for the database layer and speed up a lot of tests. Stubs are often overused and tend to complicate testing code, and interaction tests are often too implementation dependent and brittle. Does it really matter how many times a particular resource is called unless you’re implementing a cache? If you use stubs or interaction tests, it is good to also have an integration or end-to-end test that covers the real interaction.
- A comprehensive test suite requires some larger tests, both to ensure that the tests match the fidelity of the system under test, and to address issues that unit tests can’t adequately

cover. The default timeout in the build tool for large tests is an hour, but Google has tests that can take days. Since the failure can be detected far from the source, Google has annotations for test owner so you know who to contact if a large tests fails. Even large tests should be as small as possible that still ensures fidelity. Large tests are composed of a system under test, data, action, and verification. A good design includes a test strategy that identifies risks, and larger tests that mitigate them. Extra effort must be made for larger tests to not introduce friction in the developer workflow.

- You should consider good documentation part of the cost of writing software. An error in the documentation is a bug.
- Often you will find good ways to improve an API while writing the documentation for it.
- In version control we see in mono-repo and trunk based development models, that keeping branching simple generally leads to better engineering outcomes. Try to keep branches as short lived works in progress that can easily be merged back to trunk There is generally no problem with release branches, especially if your organization hasn't built up to continuous deployment where it is easy to release from trunk. But long lived dev branches are often a nightmare to merge and test. Developers should have a clear one version to commit to and depend upon.
- At Google, they often have the luxury of writing their own dependencies. It must be nice having so many software engineers you can afford to develop tangentially to your core product to make sure the job is done right. They also write their own build system which their engineers love, their own code search tool (of course, they are the "search" company), their own code review system, and even their own programming language to use heavily. Their build system sounds interesting, and they made a strong case for it over more flexible task based systems commonly in use, so I may have to try the open source Bazel project derived from it. They made a strong argument for being conservative with updates and even trying to never update dependencies, but pointed out that in real life security fixes normally force our hand, and they elaborated on the problems in semver and how they would fix it, but conceded it's good enough most of the time but that scaling really shows its weaknesses.
- It used GFlags as an example of when choosing to open source a library became a liability. It basically became an open source fork disconnected from the internal tool for a number of reasons.
- Be liberal with silencing failures. CI Failures that aren't indicative of problems in Production shouldn't block the build, and alerting that sends notifications when there isn't a Production problem should be silenced. It's far too easy to set an alert based on some abnormal statistics, but everything may be working fine despite an increase in retries. It's good to have the statistics for the post mortem after a real failure, but don't stay attached to

abnormality alerts that aren't predictive of problems and we're only set because they are easy to monitor. Failing end-to-end tests are often a problem with the tests, not a problem that you'll see in production. So it is good to automatically retry such tests, and since that makes them slower, it's nicer to have them isolated to closer to the production deployment rather than in the pre-submit process where speed is of the essence.

- Testing in a sandbox is great for reproducibility. Google has tests that spin up as many as 400 servers sandboxed to test an application.
- Tests give us confidence to change. Rollbacks give us confidence to undo. The two together lead to a productive, efficient build environment.
- Google only runs project unit tests on pre-submit and leaves other testing to the CI System. They claim code that passes unit tests is 95% likely to pass other testing.
- Google assigns a build cop on each project whose job is to drop everything and fix the build (often by rolling back) to maintain productivity.